GOVERNANÇA E COMPLIANCE AMBIENTAL

EcoCompliance API

Pipeline de CI/CD, containerização e deploy em staging e produção.

FIAP — Tecnologia em Análise e Desenvolvimento de Sistemas. Disciplina de DevOps, fase “Navegando pelo mundo DevOps”. A aplicação veio da disciplina de Java Advanced (microsserviços com Spring).

Integrantes

Nome	RM
Évelyn Santos Rodrigues	565919
Fagner Nunes Lopes	564220
Lucas Fernando da Silva	563626
Raphael Salomão Chamma	564765
Ricardo Negrão da Veiga	564106

1. Pipeline

A ferramenta é o GitHub Actions. O arquivo versionado fica em `.github/workflows/ci-cd.yml`, no repositório `github.com/fagnerlopes/ecocompliance-api`. Push e pull request nas branches **main** e **develop** disparam o fluxo.

Etapa	Quando	O que executa
Build	Todo push e pull request	mvn -B -DskipTests package com JDK 21
Testes	Em paralelo com o build	mvn -B test — 166 testes, sem banco
Somente develop	Pull request com destino main	Confere se a branch de origem é a develop
Imagem	Push em develop ou main	az acr build publica a imagem no ACR
Deploy staging	Push em develop	Atualiza o App Service ecocompliance-staging
Deploy produção	Push em main	Atualiza o App Service ecocompliance-production

No pull request rodam Build e Testes. Quando o destino é a main, entra também o job Somente develop. A publicação da imagem e o deploy acontecem no push: a **develop** atualiza o staging e a **main** atualiza a produção.

Staging: `https://ecocompliance-staging.azurewebsites.net`
Produção: `https://ecocompliance-production.azurewebsites.net`

A imagem fica no Azure Container Registry `acrecocompliance.azurecr.io`, com a tag do commit e a tag `latest`. Cada App Service passa a usar a tag do commit publicado.

2. Dockerfile

O `Dockerfile` é multi-stage. A primeira etapa usa `maven:3.9-eclipse-temurin-21`. Ela copia o `pom.xml`, baixa as dependências e gera o jar com `mvn clean package -DskipTests`. A suíte permanece no job Testes do pipeline.

A imagem final usa `eclipse-temurin:21-jre-alpine`. Ela cria o usuário `spring`, copia o jar, expõe a porta 8080 e inicia a API com `java $JAVA_OPTS -jar app.jar`. O resultado leva o JRE e o jar.

Dockerfile

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

ENV JAVA_OPTS=""
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

3. Docker Compose

O `docker-compose.yml` sobe a API e o Oracle XE 21 na rede `ecocompliance-net`. O banco usa a imagem `gvenzl/oracle-xe:21-slim`, grava os dados no volume `oracle-data` e só libera a API depois do healthcheck.

A API é construída pelo Dockerfile e recebe a tag `ecocompliance-api:${APP_ENV}`. Dentro da rede ela alcança o banco pelo host `oracle-db`, na URL `jdbc:oracle:thin:@oracle-db:1521/XEPDB1`. Porta, nome do container, usuário e `APP_ENV` vêm do arquivo de ambiente, e os dois ambientes locais convivem na mesma máquina.

Ambiente	API	Oracle no host	Arquivo
Produção	localhost:8080	1521	deploy/production.env
Staging	localhost:8081	1522	deploy/staging.env

Para subir cada ambiente:

```
./scripts/deploy-ambiente.sh deploy/production.env
./scripts/deploy-ambiente.sh deploy/staging.env
```

docker-compose.yml

```
services:
  oracle-db:
    image: gvenzl/oracle-xe:21-slim
    container_name: ${ORACLE_CONTAINER_NAME:-ecocompliance-oracle}
    ports:
      - "${ORACLE_PORT:-1521}:1521"
    environment:
      ORACLE_PASSWORD: ${ORACLE_PASSWORD:-oracle}
      APP_USER: ${DB_USERNAME:-ecocompliance}
      APP_USER_PASSWORD: ${DB_PASSWORD:-ecocompliance123}
    volumes:
      - oracle-data:/opt/oracle/oradata
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 90s
    networks:
      - ecocompliance-net

  ecocompliance-api:
    build: .
    image: ecocompliance-api:${APP_ENV:-local}
    container_name: ${API_CONTAINER_NAME:-ecocompliance-api}
    ports:
      - "${API_PORT:-8080}:8080"
    depends_on:
```

```
      oracle-db:
        condition: service_healthy
    environment:
      APP_ENV: ${APP_ENV:-local}
      DB_URL: jdbc:oracle:thin:@oracle-db:1521/XEPDB1
      DB_USERNAME: ${DB_USERNAME:-ecocompliance}
      DB_PASSWORD: ${DB_PASSWORD:-ecocompliance123}
      JWT_SECRET: ${JWT_SECRET:-Y2hhdmUtc2VjcmV0YS1lY29jb21wbGlhbmNlLWFwaS1maWFwLTIwMjUtbXVpdG8tbG9uZ2EtZS1zZWd1cmE=}
      JAVA_OPTS: "-Xms256m -Xmx512m"
    networks:
      - ecocompliance-net

volumes:
  oracle-data:

networks:
  ecocompliance-net:
    driver: bridge
```

4. Arquivos do GitHub

O pipeline do repositório está em `.github/workflows/ci-cd.yml`. O gatilho cobre push e pull request de **main** e **develop**. As variáveis do workflow apontam o registry `acrecocompliance.azurecr.io`, a imagem `ecocompliance-api` e o grupo `rg-ecocompliance`.

Build compila o pacote com JDK 21 Temurin. **Testes** roda a suíte na mesma versão do Java, em paralelo com o build. **Somente develop** entra em pull request para a main e falha quando a origem é outra branch. **Imagem** depende de Build e Testes e, no push de develop ou main, executa `az acr build` com a tag do commit e a tag latest.

Deploy staging segue depois da imagem quando a ref é `refs/heads/develop`. O job aponta o App Service `ecocompliance-staging` para essa tag e reinicia o site. **Deploy produção** faz o mesmo com `ecocompliance-production` quando a ref é `refs/heads/main`.

O login na Azure usa o segredo `AZURE_CREDENTIALS` do repositório. A proteção das branches, configurada no GitHub, exige os checks Build e Testes. Na main também exige Somente develop, inclusive para o administrador. O botão de merge permanece bloqueado enquanto um check obrigatório falha.

.github/workflows/ci-cd.yml

```
name: CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

env:
  REGISTRY: acrecocompliance.azurecr.io
  IMAGE: ecocompliance-api
  RESOURCE_GROUP: rg-ecocompliance

jobs:
  build:
    name: Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Compilar o pacote
        run: mvn -B -DskipTests package

  test:
    name: Testes
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
```

```
    - name: Executar a suíte
      run: mvn -B test

somente-develop:
  name: Somente develop
  if: github.event_name == 'pull_request' && github.base_ref == 'main'
  runs-on: ubuntu-latest
  steps:
    - name: Recusar pull request que não venha da develop
      run: |
        if [ "${{ github.head_ref }}" != "develop" ]; then
          echo "A main só aceita pull request vindo da branch develop. Esta origem é ${{ github.head_ref }}."
          exit 1
        fi
        echo "Origem aceita: develop."

imagem:
  name: Imagem
  needs: [build, test]
  if: github.event_name == 'push' && (github.ref == 'refs/heads/develop' || github.ref == 'refs/heads/main')
  runs-on: ubuntu-latest
  outputs:
    tag: ${{ github.sha }}
  steps:
    - uses: actions/checkout@v4
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Publicar a imagem no ACR
      run: |
        az acr build \
          --registry acrecocompliance \
          --image "${IMAGE}:${{ github.sha }}" \
          --image "${IMAGE}:latest" \
          .

deploy-staging:
  name: Deploy staging
  needs: imagem
  if: github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest
  steps:
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Atualizar o App Service de staging
      run: |
        az webapp config container set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-staging \
          --container-image-name "${REGISTRY}/${IMAGE}:${{ needs.imagem.outputs.tag }}" \
          --container-registry-url "https://${REGISTRY}" \
          -o none
        az webapp config set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-staging \
          --generic-configurations '{"acrUseManagedIdentityCreds": true}' \
          -o none
        az webapp restart --resource-group "$RESOURCE_GROUP" --name ecocompliance-staging

deploy-production:
  name: Deploy produção
  needs: imagem
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Atualizar o App Service de produção
      run: |
        az webapp config container set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-production \
          --container-image-name "${REGISTRY}/${IMAGE}:${{ needs.imagem.outputs.tag }}" \
          --container-registry-url "https://${REGISTRY}" \
          -o none
        az webapp config set \
          --resource-group "$RESOURCE_GROUP" \
          --name ecocompliance-production \
          --generic-configurations '{"acrUseManagedIdentityCreds": true}' \
          -o none
        az webapp restart --resource-group "$RESOURCE_GROUP" --name ecocompliance-production
```

5. Evidências

Capturas de 10/10/2026. Os dois App Services responderam em `/ambiente` e o Swagger abriu nos dois endereços. No GitHub Actions, o push da develop publicou o staging e o push da main publicou a produção. A suíte dessa execução concluiu com 166 testes e nenhuma falha.

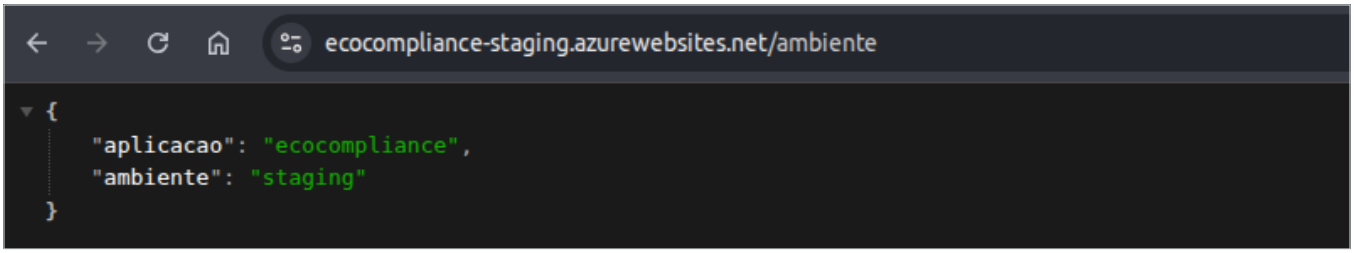


Figura 1. Staging em https://ecocompliance-staging.azurewebsites.net/ambiente.

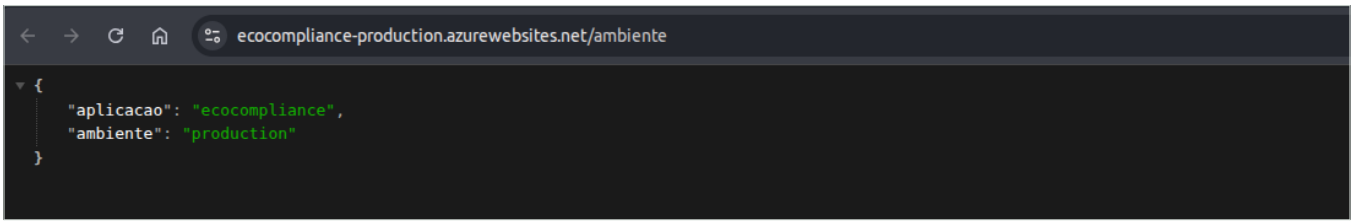


Figura 2. Produção em https://ecocompliance-production.azurewebsites.net/ambiente.

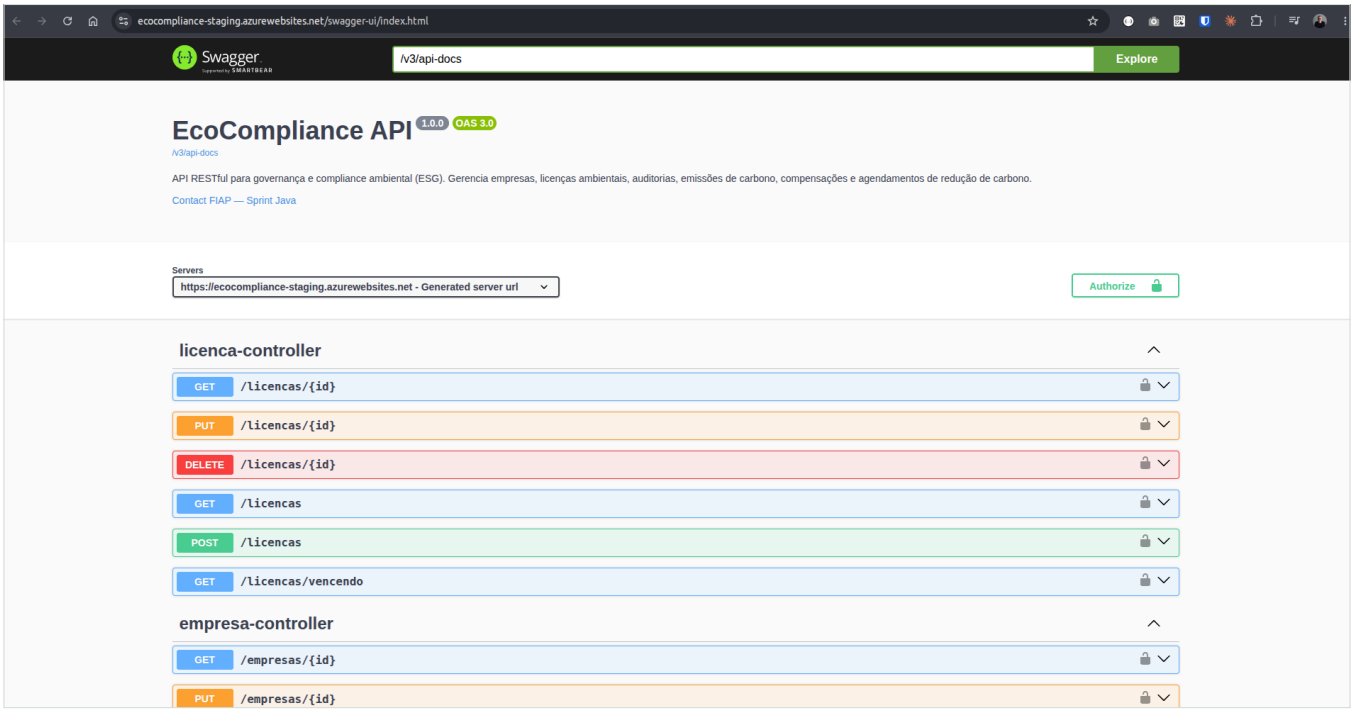


Figura 3. Swagger do staging em https://ecocompliance-staging.azurewebsites.net/swagger-ui/index.html.

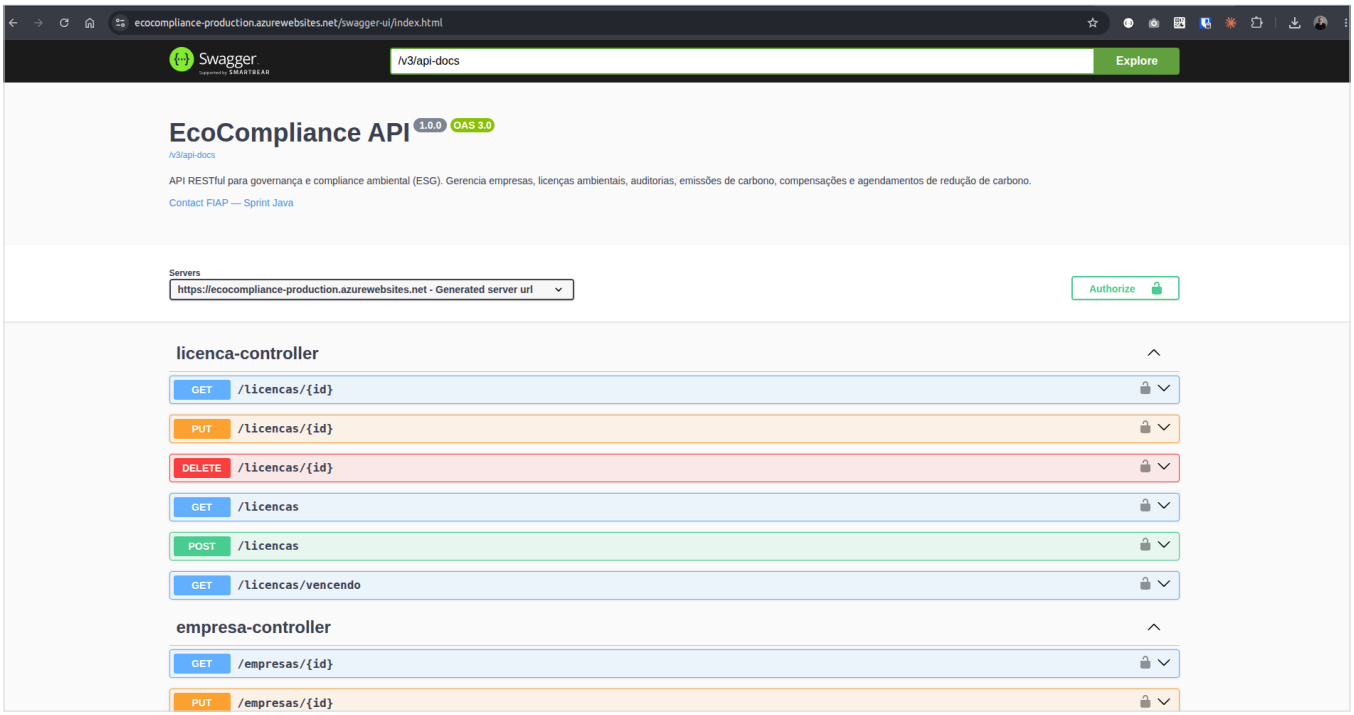


Figura 4. Swagger da produção em https://ecocompliance-production.azurewebsites.net/swagger-ui/index.html.



Figura 5. Push na develop: Build, Testes, Imagem e Deploy staging concluídos. Somente develop e Deploy produção ficam ignorados nesse evento.

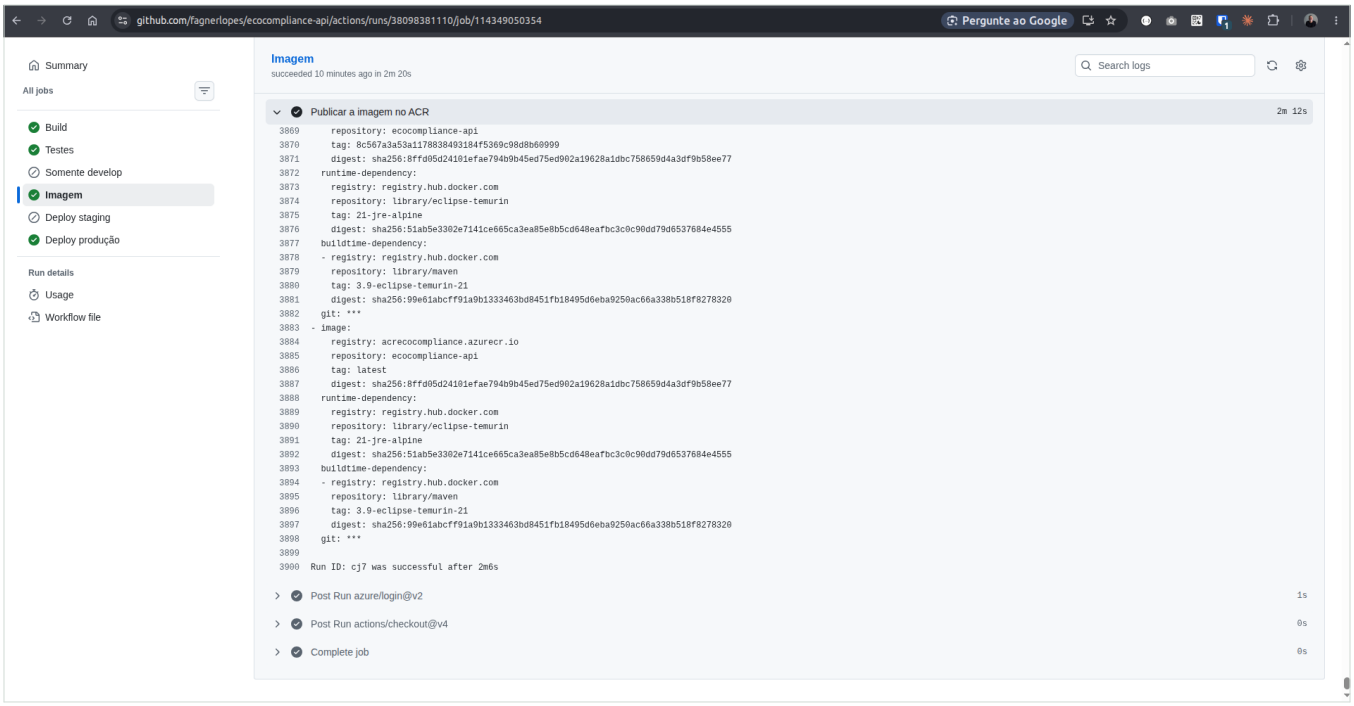


Figura 6. Push na main: Build, Testes, Imagem e Deploy produção concluídos. O job Imagem publica a imagem no ACR acrecocompliance.azurecr.io.

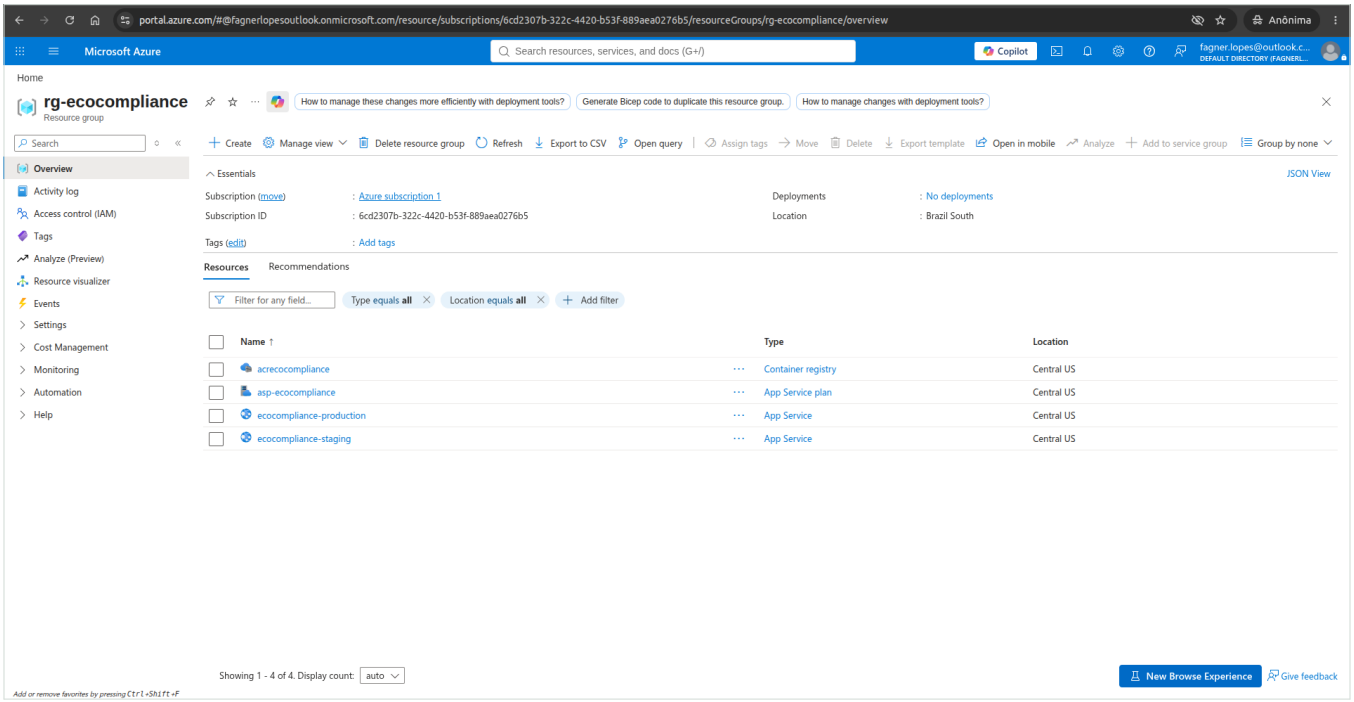


Figura 7. Grupo rg-ecocompliance: o registry acrecocompliance, o plano asp-ecocompliance e os App Services de produção e staging.

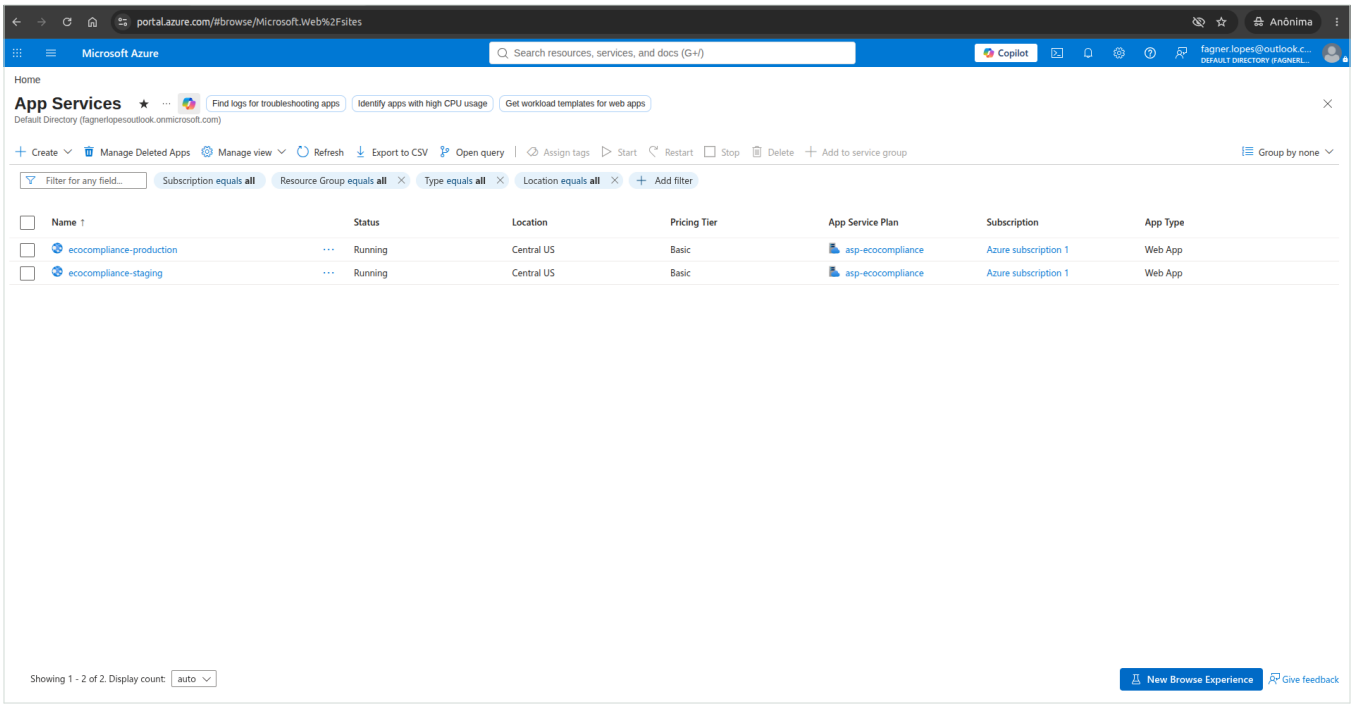


Figura 8. Os dois App Services no plano asp-ecocompliance, região Central US.

6. Desafios

- Staging e produção locais usam projeto, volume, rede e porta diferentes. Encerrar um ambiente preserva o banco do outro.
- A construção da imagem gera o jar e a etapa final leva o JRE. A suíte permanece no job anterior à publicação da imagem.
- O mesmo workflow atende os dois ambientes. A branch decide o destino: develop publica o staging e main publica a produção.
- A main recebe pull request da develop. O job Somente develop e a proteção da branch sustentam essa regra, também para o administrador do repositório.

7. Checklist

Item	OK
Projeto compactado em ZIP com estrutura organizada	sim
Dockerfile funcional, com explicação e exemplo	sim
docker-compose.yml, com explicação e exemplo	sim
Pipeline com build, teste e deploy	sim
Arquivo do GitHub Actions documentado	sim
README.md com instruções e evidências	sim
Documentação técnica (este PDF)	sim
Deploy em staging e produção	sim